Best Model for FMA Audio Genre Classification

Christian Kurdi

CS460

5/3/26

Abstract:

Classification of the genre of an audio sample can occur by use of many machine learning models, but which model performs the best with the FMA dataset for audio analysis needs to be evaluated. There are multiple subsets within the FMA dataset, but for the purpose of this research only the small subset of samples and MFCC values are considered for classification. A support vector machine model (SVM), random forest model, XGBoost model, k-Nearest Neighbor (KNN) model, and a neural network model are all analyzed in this report, along with PCA dimensionality reduction, standardization, and hyperparameter tuning with grid search cross-validation along with use of accuracy, F1 macro, and balanced-accuracy metrics. Although each model performs well in their own scenarios, results show that KNN models classify genre the best when the input data includes MFCC values of audio samples. This doesn't mean scores are necessarily "good", but they are the best from the working models. There are more features for testing as well, but in the given environment for this project, KNN is the model to use for classification.

Introduction:

Problem Definition – To classify the genre of a song

Dataset Description – The dataset used for this report is the FMA dataset for Music Analysis. Created by Michaël Defferrard, Kirell Benzi, Pierre Vandergheynst, and Xavier Bresson, the Free Music Archive (FMA) provides 917 GiB of audio tracks and datasets to accomplish making large audio datasets available for browsing, searching, and organizing. The features of audio samples include Mel-Frequency Cepstral Coefficients (MFCC's), among other subsets. MFCC's transform raw audio into integers representing how humans hear sound, which is more useful when used with machine learning models. The target variable of the dataset is the genre for a given track. Multiple genres can exist for one track, but for the sake of this project only the small dataset is in use which contain only single genres for each track. The features dataset contains 106,574 instances and 518 features, and the tracks dataset contains 106,574 instances and 52 features.

Objective – Which machine learning model is best to provide the genre for an audio sample by using the MFCC's? What parameters work best with the highest-ranking model?

Report Structure – The remaining sections of this report explain my approach to answering my objectives, provide more details on important concepts existent in the research, analyze results, and propose future avenues for exploration regarding better scores and models.

Exploratory Data Analysis:

My initial steps for analyzing the data began with just being able to view the data from the dataset. Through this, I noticed there were many “NaN” values which needed to be addressed specifically because some genres were listed as “NaN”. I also noticed a lot of features for tracks were redundant in regards to genre classification; there was a lot of metadata which could be ignored. I also created some histograms for MFCC’s to view distribution of the data in hopes I would find outliers, some of these can be seen in figure 1A and 1B below. The broad view of several histograms shows that some of the data is well grouped, others have a large deviation. Figure 1B shows specific histograms for the given MFCC values. When you get a closer look, the data is very spread out for some histograms.

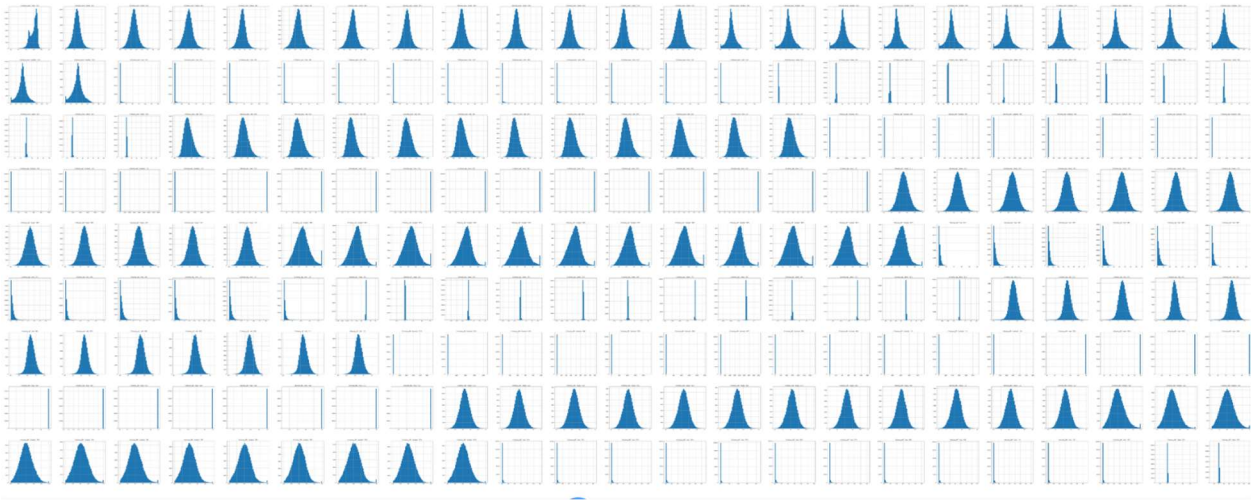

Figure 1A

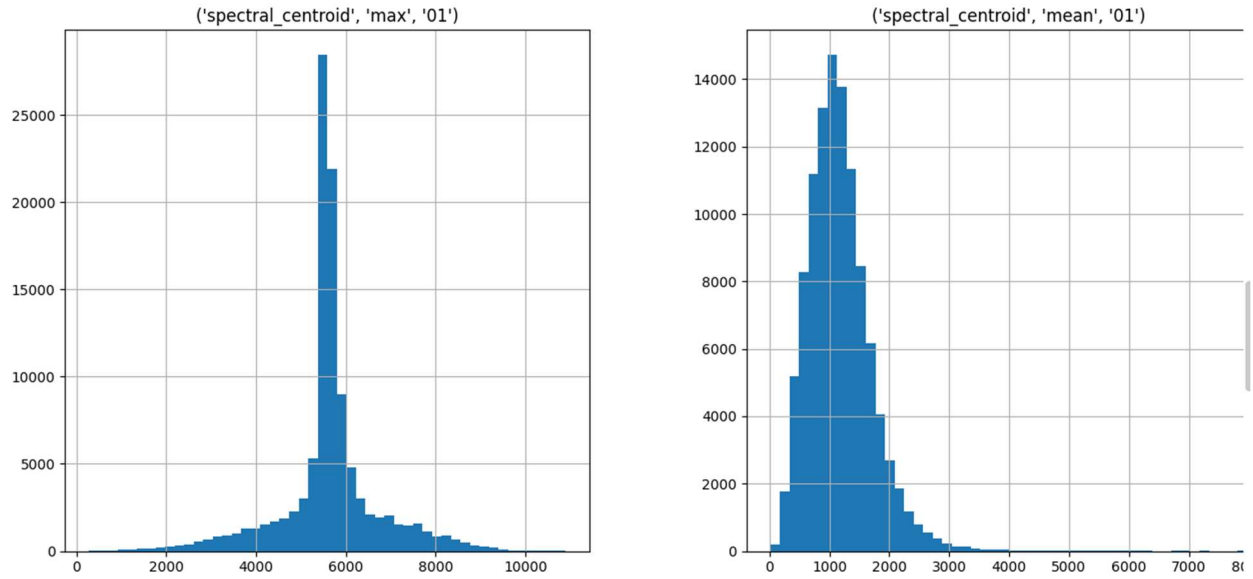

Figure 1B

Through the plots, I observed that the data needed to be standardized due to a wide range of values making it hard to accurately depict the values. It was also necessary to reduce dimensionality of the features due to the multi-indexing from the FMA dataset. This requires PCA dimensionality reduction. I then decided to find correlations between the MFCC's. From figure 2 below, you can see positively correlated MFCC values in shades of red and negatively correlated MFCC values in shades of blue.

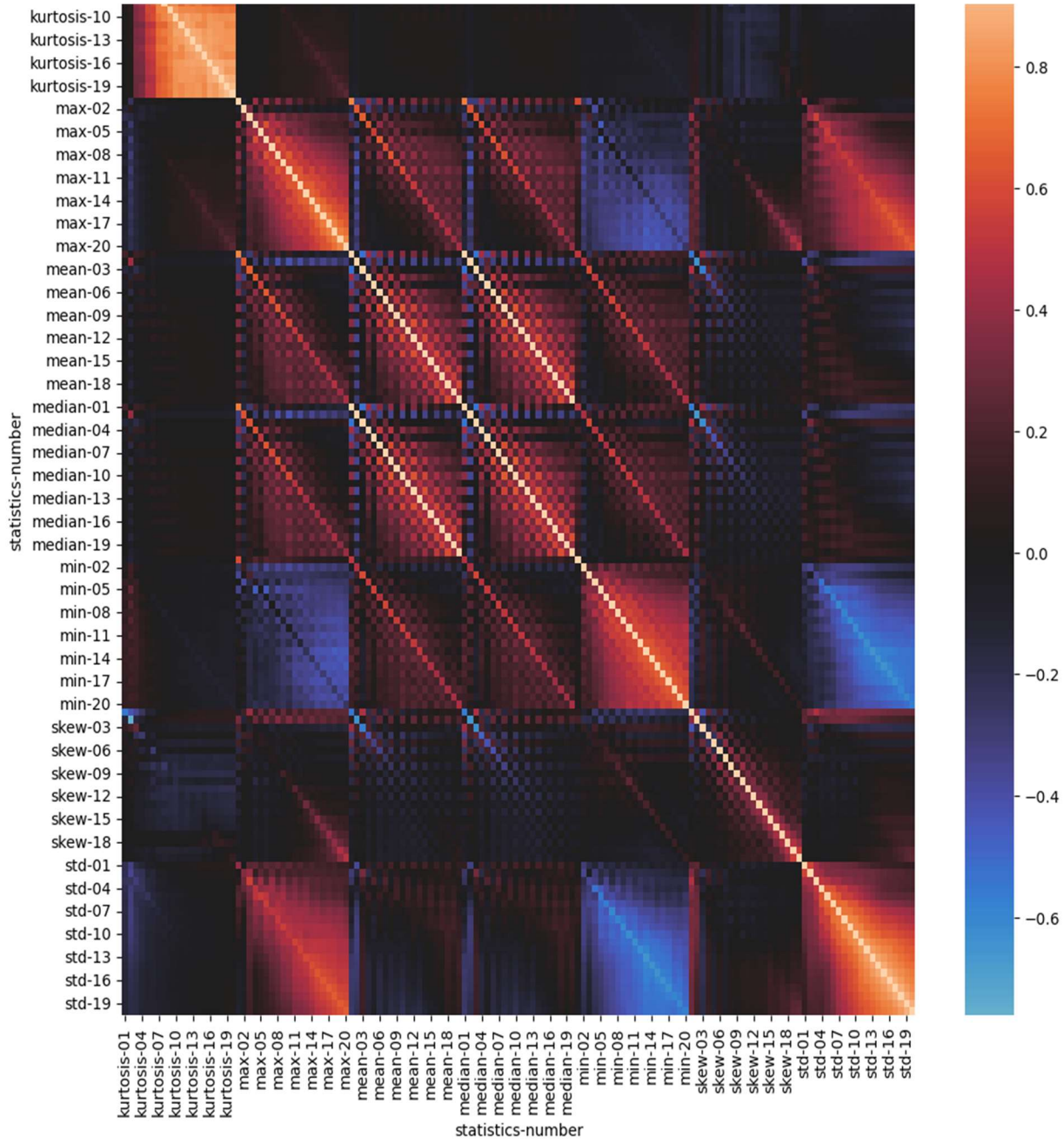

Figure 2

Methodology:

When it came to preprocessing, a few things needed to occur due to my findings from the exploratory data analysis stage. I started by replacing the “NaN” values for tracks

with no genre data. This mostly was to allow the models to be able to utilize the data as well as make predictions. I chose to replace all “NaN” values with the string “none” due to the other genre labels being a string, as well as this being required for proper label encoding used in some models. Using the “none” label allows for me to still manage which tracks don’t have valid genre’s while allowing the model to utilize all rows of the dataset.

The FMA dataset automatically includes values for tracks which should be within training, validation, and test sets. This is helpful due to the need for a balance of genre’s within the training and test sets, as well as keeping all tracks from an artist within the same set. By having all tracks from an artist in the same set, the models are not able to memorize genre’s of a particular artist from both the training and test sets. With them in only the training or the test set, the model is forced to predict a genre for an unknown artist with reliable results based on the MFCC data alone.

With the training and test sets established, and more findings in the exploratory data analysis stage, I decided to standardize the data. The need for this is due to the data having a large range of values. By standardizing the data, the large values are transformed into a range similar to the other values. This is because the attributes are transformed so that they are distributed to a standard normal distribution with a mean of 0 and a standard deviation of 1. This helps during visualization of data, as well as performance of models.

I also decided to utilize Principal Component Analysis (PCA) to reduce dimensions of the dataset. The FMA dataset utilizes multi-indexing to organize both the tracks dataset as well as the features dataset. This enhances organization of all the data while keeping the necessary data associated with its track. Because of this, the FMA dataset has a very large degree of dimensions which requires PCA for dimension reduction. I chose 95% retention of variance so that PCA keeps the smallest number of components required to meet 95% variance, meaning most of the information is retained while still using less dimensions. This greatly reduced the execution time of model training due to the reduced dimensions.

The models I selected revolve around the need for reliable interaction with audio features. The first model I selected is a Support Vector Machine (SVM), which are supervised learning methods useful with classification. I specifically chose linear support vector classification (SVC) from Sklearn because it performs faster than regular SVC with large datasets such as FMA. Linear SVC “has more flexibility in the choice of penalties and loss functions and should scale better to large numbers of samples” (scikit-learn developers, 2026). SVC works by finding a hyperplane that separates classes which maximizes the margin from the closest points. 3-fold cross-validated grid search was also used to find the best parameters for SVC, with loss function being varied as well as the maximum number of iterations.

The second model I chose is a Random Forest classifier. This model handles noisy features well, meaning features that have little to no correlation with the target (the genre for the track) don't affect the model as much. This model would also work very well if all features of the FMA dataset were taken into account, such as chroma, along with MFCC. Chroma is essentially the 12 semitones that make up an octave, and the representation means the intensity of the chroma at each time frame. For the sake of execution time across 6 models, chroma was not utilized in training or predictions. A random forest model creates many decision trees with different features and data. Whatever genre is the majority of predicted genres out of all trees decides the final prediction for the random forest model.

I used 3-fold cross-validated grid search for tuning hyperparameters. I varied the number of estimators to see if the number of trees in a forest affects accuracy. Using accuracy scores allowed me to see how the parameters are affecting the model. I also used F1 score, and balanced-accuracy score since they work with imbalanced data.

The third model I chose is an XGBoost classifier. Similar to both SVM and Random Forest, XGBoost works well with this dataset because it is noise tolerant and can handle the large amount of features associated with audio analysis. XGBoost can also handle imbalanced data better than most models, which would improve usability as the dataset becomes even larger. XGBoost models work similarly to random forest models in that many decision trees are made, except that each new tree solves the problems with the previous tree created. The trees are trained sequentially, whereas random forest trains independent trees.

Hyperparameter tuning was performed through 3-fold cross-validated grid search. I chose to vary the number of estimators, the learning rate, and the maximum depth to find which provides the best score. Execution time was very long during these tests, so I only tried 2 values to test for each parameter stated above. The accuracy score allowed me to get an idea of how the model is performing and how the changes in parameters affect its baseline performance. I also utilized F1 macro scoring, as well as balanced-accuracy scoring with these parameters. F1 macro, in this case, is helpful because it performs well with multi-class classification which occurs with the many different types of genres a track can be classified as. A balanced-accuracy score works better than a plain accuracy score because if classes are imbalanced a plain accuracy score is not actually accurate. I wanted to see if this metric would provide a better score than plain accuracy as well as F1 because it has similarities to both, only that it uses the average of recall for each class instead of F1 and plain accuracy.

The fourth model included with this report is a k-Nearest Neighbor (KNN) classifier. This is a great model to use with the FMA dataset because audio features automatically cluster naturally, which makes it easy to integrate with the clusters of KNN. Since similar points are close together, this model makes it easy to predict genres for tracks with similar features. Hyperparameter tuning was performed with 3-fold cross-validated grid search. A KNN model tunes no parameters by itself, but it autonomously finds classes within the data and utilizes proximity functions for prediction.

The fifth model, a neural network, is implemented using Scikit-learn's multi-layer perceptron (MLP) classifier model. While performing the initial training with the model, the maximum iterations were reached before finishing. This prompted me to utilize the parameter for maximum iterations in the tuning phase, also done with grid search. I also varied the solver parameter which is involved in weight optimization. A neural network is composed of neurons, and each has a nucleus which is a function on input, inputs which are weights and the bias value, and an output or predicted value. The more neurons in a network, the more likely the model can find a function to accurately fit the data.

Regarding the data splitting of the data set, the creators provided subsets which already belong to train, test, and validation sets. This is due to the need for balanced numbers across genres, since imbalanced numbers will skew metric scores for models. These sets were then split into X and y values using features for X sets, and genre's being the target (y) values. I chose to use the MFCC values for the audio samples for determination of genre due to them being a specific descriptor for audio characteristics.

Across all models, a confusion matrix was created to view accuracy based on actual and predicted values. By analyzing predictions this way, it's easy to see which predictions are occurring correctly and which are giving poor results. False positives and negatives show where the model is incorrectly classifying genres, and true positives and negatives show where the model is performing correctly.

Results and Discussion:

The SVM model provided an initial accuracy score of 59.32%. After using grid search cross-validation to find the best parameters for the model, I got an accuracy score of 59.32% showing that the initial parameters proved to be the best parameters for the model. Using the score from F1 macro, the model has a score of 12.39%. I believe this to be caused by the data being imbalanced, with a solution to this being the existence of more data or more preprocessing. The balanced-accuracy score was 13.13%, slightly above the

F1 score. Although these scores are lower than plain accuracy, they highlight potential issues using the plain accuracy score since there is a large deviation between the scores.

The random forest model provided an accuracy score of 59.80% without any parameter tuning. After performing cross validated grid search, I found the best parameters which provided an accuracy score of 59.71%. This is lower than before the parameter tuning, meaning the model got less accurate after finding the best parameters. This also means the model found a better distribution of classes causing the lower accuracy score. I then calculated the F1 macro score using the best parameters and received a 13.07% score. This large drop is similar to the balanced-accuracy score of 12.37%. Both scores are worse than basic accuracy, indicating the dataset may require more data to balance the classes.

The XGBoost model provided scores which I was not expecting. Before any parameter tuning, using the basic accuracy score I got a result of 60.46% accuracy which is similar to accuracy of the previous models. After finding the best parameters through grid search cross validation, I ran the accuracy score again and improved slightly to a 60.81% score. I wanted to see if other scoring metrics would improve scores, and chose F1 macro and balanced-accuracy. The average F1 score is measured with every class treated equally, meaning genres with more samples than other genres are still treated the same in terms of weight. The F1 score produced when I ran the model was only a 19.52% which is significantly less than the accuracy score. This could be due to the balanced weight of classes, so if the model has smaller classes for genres it's more likely to fail on these which reduces the overall score for the model. The model does well with large, well populated classes but poorly with smaller groups of classes. This low score could also be due to not many different hyperparameters being tested by the grid search cross validation, so the model may be overfitting or underfitting the data. Balanced-accuracy is like F1 in that classes are weighted equally regardless of samples belonging to each class. After performing the scoring test, I found similar results to the F1 score, a 17.47% metric. This is actually slightly worse than F1, possibly due to potential flaws like that with F1 macro calculations. If the recall is low for certain genres, the overall balanced accuracy score is low as well meaning a few genres can lower the average even if other genres provide good recall values.

The scores for the k-nearest neighbor model proved valuable, with 49.29% given for the initial accuracy score. This is the lowest initial score out of the models up to this point. After parameter tuning, the accuracy changed to 58.45%. This showed the greatest increase in score due to parameter tuning. Because of the cross-validation grid search, out of the tested parameters the model performs best when the nearest neighbors parameter

is 15 and the weight function used in prediction is “distance” meaning the weights use the inverse of their distances. The F1 macro score turned out to be equally disappointing as the other models, being at 20.37%. Similarly low is the balanced-accuracy score at 19.95%. Once again, these low scores are due to the dataset in use meaning further data cleaning and/or preprocessing is required for better results. In relation to the other models, both the F1 score and the balanced-accuracy score for KNN are the highest so far.

In regards to the neural network multi-layer perceptron classifier the initial accuracy was given as 49.29%. However, this should not be treated as accurate because the maximum number of iterations was reached for the model and it terminated before it converged. After performing the hyperparameter tuning with larger iterations, I found the execution time became more long than feasible for the length of this project so unfortunately the scores for neural network aren’t as accurate as they would be if the model could have the number of iterations it would need to converge. Because of this, the neural network model is not considered a candidate for best model although results will still be analyzed. The score after tuning was 60.14%.

Once I had all metric scores for each model I computed the average of only scores from the best hyperparameters per model, and chose the best model being the one with the highest average score. From this, I concluded that the KNN model is the best model for genre classification with this dataset by a slight margin over XGBoost. You can see this data in Figure 3 below.

Model	Initial Accuracy	Tuned Accuracy	F1 Macro	Balanced-Accuracy	Average
SVM	59.32%	59.32%	12.39%	13.13%	28.28%
Random Forest	59.80%	59.71%	13.07%	12.37%	28.38%
XGBoost	60.46%	60.81%	19.52%	17.47%	32.60%
KNN	49.29%	58.45%	20.37%	19.95%	32.92%
Neural Network	*49.29%	*60.14%	*20.92%	*20.71	*33.92%

Figure 3

	Blues	Classical	Country	Easy Listening	Electronic	Experimental	Folk	Hip-Hop	Instrumental	International	Jazz	Old-Time / Historic	Pop	Rock	Soul-RnB	Spoken	none
Blues	0	0	0	0	0	0	0	1	0	0	0	0	0	4	0	0	8
Classical	0	31	0	0	1	8	1	0	1	1	0	2	0	0	0	0	42
Country	0	0	1	0	0	0	1	3	0	1	0	0	0	0	0	0	12
Easy Listening	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	5
Electronic	0	4	1	0	216	35	4	38	3	3	2	0	2	25	1	1	504
xperimental	0	12	0	0	43	152	12	24	15	4	1	0	4	90	0	1	727
Folk	0	4	2	0	4	15	21	3	0	7	1	1	3	37	0	0	201
Hip-Hop	0	0	1	0	29	2	1	152	1	1	0	0	1	8	0	0	127
nstrumental	0	11	1	0	6	24	4	0	6	0	1	0	1	24	0	0	231
nternational	1	1	2	0	5	9	5	5	0	17	0	0	0	5	0	0	78
Jazz	0	1	1	0	0	7	0	0	1	0	2	0	0	3	0	0	32
Old-Time / Historic	0	0	0	0	0	1	0	0	0	0	0	52	1	0	0	0	1
Pop	0	0	0	0	21	5	7	6	1	1	0	0	1	24	0	1	137
Rock	0	3	3	0	35	33	9	16	5	7	2	0	13	782	0	4	552
Soul-RnB	0	0	0	0	9	2	0	0	0	1	0	0	0	9	0	0	22
Spoken	0	0	0	0	1	11	2	2	0	1	0	0	0	3	0	4	7
none	0	112	11	4	552	371	124	208	55	50	24	8	35	627	3	13	4115

Figure 4

Figure 4 shows the confusion matrix for the KNN model. The rows represent the actual genres, and the columns represent the predicted genres. If (row, column) is (row, row), meaning the same class for both axis, that location is a correct classification. Also, if the value at (row, column) is the same as (column, row), the model is classifying correctly.

I think the KNN model performed the best because the clustering performed by the model is similar to the natural clustering that occurs in audio features. It started with the lowest accuracy score, but responded the best to parameter tuning. It then also got the highest F1 and balanced-accuracy scores out of all the tested models. Also, the execution time for this model was significantly faster than every other model, which further iterates the idea that it fits well with this type of dataset.

Unfortunately, there are several limitations to my approach to this research. Without being able to cross-validate a significant number of times, or use many different parameters for each model, improvements found for best parameters may not have been the actual best parameters for the model with this dataset. I also only considered MFCC values, when other values such as chroma or kurtosis also contain weights for classification of audio samples.

Conclusion:

Through research and analysis of different models with the FMA dataset, it was possible to determine the best model with the best parameters for classification of genres. By testing multiple machine learning models for classification and applying scores for measurement of the models, statistics were found indicating that a KNN model with 15 nearest neighbors, distance for the weight function parameter used in prediction, and default values for the rest of available parameters provide the best classification with this dataset. The results found by testing many different models provide insight into which models work well in certain scenarios with specific parameters and create further opportunities for exploration in research with differences in datasets, audio descriptor values, and different targets for prediction aside from genres.

There are many other parameters I'd like to have tested and measured, including different tuning methodologies. I only performed cross-validation grid search with 3 folds to find the best parameters per model. It would be worth exploring a different number of folds, or another tuning method entirely so that all results could be compared. I also only used the smallest of subsets within the FMA dataset, which undoubtedly negatively affected scores across models. I also only split the dataset according to the creator's specified descriptions within the dataset, which may or may not provide the best split for the given data. Considering future work regarding the topic of this project, the previously explained changes can be explored. New models, more parameters tested, and more scores compared may provide higher results for classification. I also think it would be worth looking into one of the poor performing models such as SVM and finding out what factors are causing poor scores. Focusing on one model and analyzing the dataset more to generate quality results would be worthwhile in the long run, since these changes can then be applied to other models to see how their scores change. This would require further exploratory data analysis and preprocessing techniques. Allowing the program enough time to execute many iterations for the neural network model would also be worth looking into to get reliable scores which can then be used to compare to existing scores.

By analyzing multiple machine learning models which classify audio samples with genres, a best model can be found to provide valuable predictions. Different models behave differently for a reason; they each serve a purpose under certain conditions. There is no right or wrong model until analysis of a dataset proves otherwise (aside from regression or classification). Through exploration, preprocessing, parameter tuning, metrics, and a lot of testing, a lot of information can be found just through data and some processing prowess. This information can then be turned into valuable and potentially life changing decisions.

Resources

Defferrard, M., Benzi, K., Vandergheynst, P., Bresson, X. (2020, May 29). FMA: A Dataset For Music Analysis. GitHub. https://github.com/mdeff/fma

scikit-learn developers. (2026). LinearSVC. https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html

scikit-learn developers. (2026). RandomForestClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

scikit-learn developers. (2026). confusion_matrix. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html

scikit-learn developers. (2026). KNeighborsClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

scikit-learn developers. (2026). MLPClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html#sklearn.neural_network.MLPClassifier

xgboost developers. (2025). Python Package Introduction. https://xgboost.readthedocs.io/en/release_3.2.0/python/python_intro.html#setting-parameters

ScienceInsights (2026, March 9). *What Is MFCC in Audio and Speech Processing?* . https://scienceinsights.org/what-is-mfcc-in-audio-and-speech-processing/

Dan Ellis. (2007, April 21). Chroma Feature Analysis and Synthesis https://www.ee.columbia.edu/~dpwe/resources/matlab/chroma-ansyn/